

HDF5 Audit Report

[NN]

Very abstract...

hello

Acknowledgments: This material is based upon work supported by the U.S. National Science Foundation under Federal Award No. 2534078. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Revision History

Version Number	Date	Comments
v1	July 4, 2026	Initial draft released for internal review.
v2	August 1, 2026	Public release.

Contents

1 Introduction 5

1.1 Purpose 5

1.2 Scope 5

1.2.1 Safety 5

1.2.2 Security 5

1.2.3 Privacy 5

2 Motivation 5

2.1 Problem Statement 5

2.2 Goals 5

3 HDF5 Core Library and File Format 7

3.1 Accidents Waiting to Happen – HDF5 Safety 7

3.2 Verify and Verify – HDF5 Security 7

3.3 Footprints, Fingerprints, Controls – HDF5 Privacy 7

3.3.1 Privacy-Sensitive Assets 8

3.3.2 Exposure Mechanisms 9

3.3.3 Lifecycle Controls 10

3.3.4 Control Levels 10

3.4 HDF5 Tools 11

3.5 Audit Findings and Mitigation Priorities 11

4 HDF5 Extensions 13

4.1 Overview 13

4.2 Virtual Object Layer (VOL) Connectors 13

4.3 Filters Plugins 13

4.4 Virtual File Drivers (VFD) 13

4.5 Audit Findings and Mitigation Priorities 13

5 HDF5 Wrappers and Language Bindings 14

5.1 Overview 14

5.2 Python 14

5.3	Java	14
5.4	Audit Findings and Mitigation Priorities	14
6	HDF5 in the Software Supply Chain	15
6.1	Upstream Dependencies	15
6.1.1	Dependency Source Modes	15
6.1.2	Baseline Build	16
6.1.3	Feature Dependencies	17
6.1.4	Filter Dependencies	20
6.1.5	Virtual File Drivers and External Connectors	21
6.1.6	Tools, Tests, and Security Features	23
6.1.7	Developer and Packaging Dependencies	24
6.1.8	External Source Defaults	26
6.2	Downstream Dependencies	28
6.2.1	High-Impact Downstream Families	28
6.2.2	Representative Downstream Projects	30
6.2.3	Audit Implications	31
6.3	Audit Findings and Mitigation Priorities	32
7	Independent HDF5 Implementations	33
7.1	Overview	33
7.2	Audit Findings and Mitigation Priorities	33
8	Long-term Data Accessibility and Interpretability	34
8.1	Audit Findings and Mitigation Priorities	34
9	Resources	35
	References	36

1 Introduction

1.1 Purpose

1.2 Scope

Out of scope:

- PHDF5
- HDFView
- HSDS
- AI

1.2.1 Safety

1.2.2 Security

1.2.3 Privacy

2 Motivation

2.1 Problem Statement

2.2 Goals

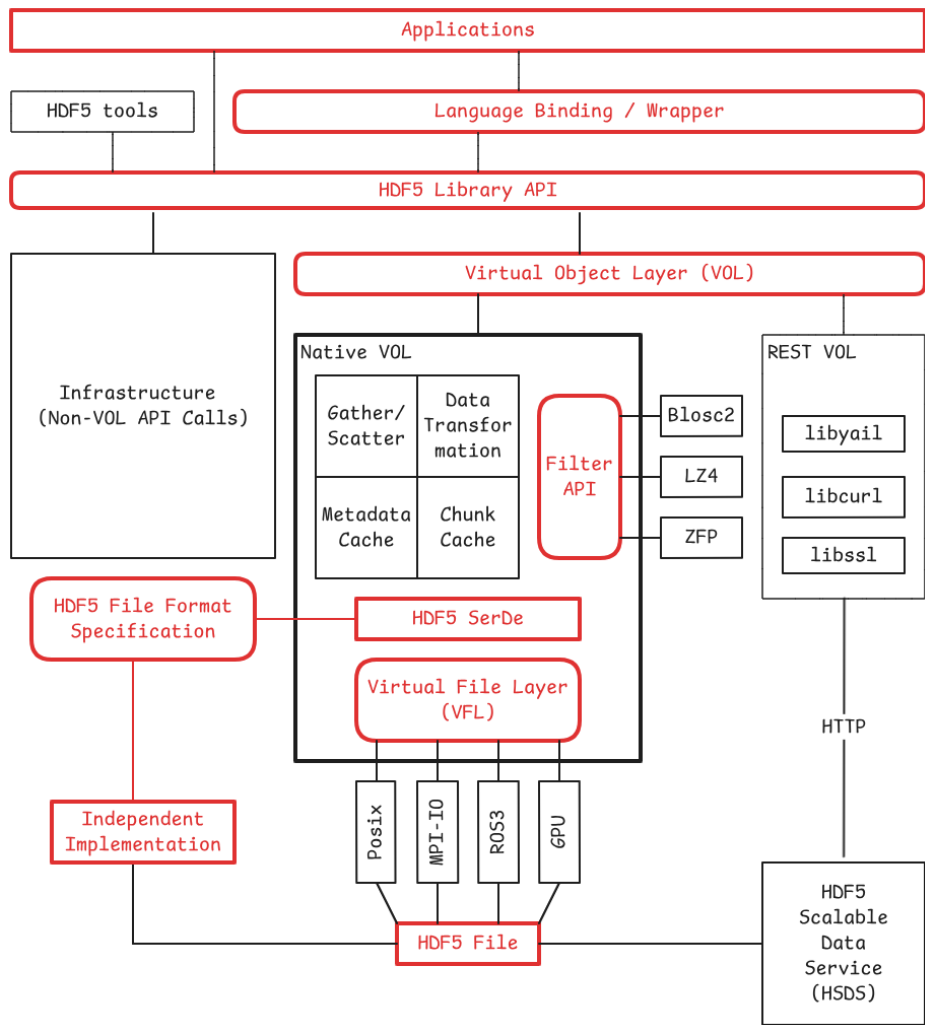


Figure 1 – HDF5 vulnerability sites (interfaces: rounded rectangles)

3 HDF5 Core Library and File Format

3.1 Accidents Waiting to Happen – HDF5 Safety

- Disk full [\[5\]](#)
- File corruption [\[3\]](#)
- Long-term data accessibility and interpretability, see section [8](#)

3.2 Verify and Verify – HDF5 Security

- Integer arithmetic
- File format parsing issues
- Malicious plugins, see section [4](#)

3.3 Footprints, Fingerprints, Controls – HDF5 Privacy

HDF5 privacy is best treated as an exposure-management problem, not only as a confidentiality or access-control problem. In a broader privacy-engineering sense, ordinary data processing can create privacy harm through inference, aggregation, secondary use, contextual change, or surveillance effects even when systems are secure and operating as designed. That distinction matters for HDF5 because the format was built to make scientific data portable, self-describing, reusable, and long-lived. Those same properties can preserve or reveal information that a data producer did not intend to disclose.

In scientific settings, sensitive information may appear outside the primary dataset values. HDF5 files often cross institutional boundaries, enter public repositories, move into cloud workflows, and survive long enough to be reused with analysis techniques that did not exist when the files were created. Machine-learning and cross-repository analysis make this more acute: information that appears harmless in one file can become revealing when correlated with quality-control products, timestamps, geospatial metadata, workflow records, or external data sources.

3.3.1 Privacy-Sensitive Assets

The privacy surface of an HDF5 ecosystem includes several asset classes.

Data values. Datasets may contain biomedical observations, wearable-sensor measurements, geospatial products, industrial telemetry, facility observations, images, simulations, or other measurements that directly disclose sensitive information or enable sensitive inference.

User metadata. HDF5 groups, datasets, committed datatypes, dataspace, attributes, object names, link names, compound datatype member names, enumeration labels, and object comments can expose context. Timestamps, coordinates, instrument identifiers, acquisition settings, calibration details, experimental identifiers, and provenance fields are scientifically useful, but may also identify people, facilities, locations, schedules, or research objectives.

Structural metadata. The hierarchy of groups and datasets, dataset dimensions, chunking, filtering pipelines, external storage, links, object references, region references, and virtual datasets can reveal categories, processing stages, cohorts, workflow assumptions, and relationships among observations even when raw values are not inspected.

Internal format metadata. HDF5 library and file-format structures can also leak clues. Examples include global heaps that store variable-length data, object reference counts, textual datatype descriptions produced by applications, creation order tracking, chunk indexing structures, and other implementation details that reveal the shape or semantics of stored content.

Derived and reconstructed information. Privacy-sensitive facts may be inferred from combinations of datasets, linked files, external links, region references, virtual dataset mappings, masks, quality-control arrays, and provenance records. In these cases the protected asset is not a single stored object, but a relationship that can be reconstructed.

Workflow artifacts. Tool output, `h5dump` reports, diagnostic logs, temporary files, intermediate products, metadata snapshots, conversion artifacts, and

plugin-generated metadata may persist after the workflow that created them. These artifacts can expose information that users do not associate with the released HDF5 file itself.

3.3.2 Exposure Mechanisms

Five exposure mechanisms recur in HDF5 workflows.

Metadata exposure. Because HDF5 is self-describing, metadata remains visible and useful across tools. That visibility can reveal acquisition time, location, instrument identity, laboratory conditions, patient class, operational status, or internal annotations even when primary data values are transformed, anonymized, or protected elsewhere.

Structural inference. File structure often mirrors scientific intent. Paths, group organization, dataset dimensions, chunking choices, compression filters, external storage mappings, and virtual dataset relationships can disclose experimental design, processing stages, classifications, or the absence or presence of expected observations.

Raw-representation exposure. Low-level storage patterns and convenience metadata may reveal information about raw content without requiring full application-level interpretation. Fixed-length strings, textual datatype descriptions, descriptive attributes, and implementation-specific layout choices can expose semantics even when direct value access is restricted.

Unintended inclusion or alteration. Converters and processing pipelines frequently enrich data to improve interoperability. Well-intentioned conversion from non-self-describing formats into HDF5 can turn implicit structure into explicit datasets or metadata. Derived products and provenance added for reproducibility can become new privacy surfaces.

Cross-data linkage. HDF5 supports linkage across files and systems through external links, external storage, virtual datasets, VFDs, and VOL connectors. These features are valuable for science, but can also make otherwise benign

measurements revealing when combined with geospatial data, timestamps, masks, quality-control arrays, or external repositories.

3.3.3 Lifecycle Controls

Privacy in HDF5 is not a static property of a file. It changes as data are collected, converted, annotated, transformed, shared, archived, replicated, and reused. A privacy-aware HDF5 workflow should therefore treat exposure review as part of the data lifecycle.

- Classify datasets and metadata before release, including attributes, names, datatype fields, dimensions, VDS mappings, references, and external links.
- Minimize metadata that is not needed for the intended reuse, and replace sensitive names, labels, locations, identifiers, and timestamps with controlled or generalized values where scientific meaning permits.
- Review conversion and enrichment tools for metadata they add automatically, especially when translating from less self-describing formats into HDF5.
- Separate sensitive provenance, operational notes, logs, and temporary workflow products from public data products unless they are intentionally part of the release.
- Treat structure as inspectable information. Redaction plans should consider object paths, hierarchy, dimensions, chunk layout, filters, links, references, and related side files, not only dataset values.
- Reassess older HDF5 archives when data are moved to new repositories, combined with other collections, opened to AI/ML workflows, or released to a broader audience than originally expected.

3.3.4 Control Levels

The practical control posture falls along a continuum.

Open by design. Some HDF5 data products are intended to be fully inspectable. For these releases, the control is explicit documentation: state what metadata, structure, provenance, links, and workflow artifacts are intentionally exposed, and verify that no unintended sensitive context remains.

Exposure-reduced sharing. The middle ground is usually the most important case. Preserve enough HDF5 metadata for reproducibility and interoperability, but remove, generalize, tokenize, or separate fields that identify people, facilities, precise locations, operational schedules, unpublished scientific objectives, or sensitive workflow details.

Strong protection. When either values or metadata cannot be safely exposed, use access control, encryption, repository policy, or container-level protections around the file and its side artifacts. Encryption of raw values alone is insufficient if names, attributes, structure, logs, links, or external storage mappings still reveal the sensitive context.

3.4 HDF5 Tools

Not strictly part of the core library, but the tools are an important part of the ecosystem and have their own set of issues.

3.5 Audit Findings and Mitigation Priorities

- HDF5 privacy review should cover user-controlled metadata and internal structural metadata in addition to raw datasets.
- Tooling should help users inventory attributes, names, references, external links, VDS mappings, filters, datatype fields, and workflow artifacts before publication.
- Documentation and training should teach HDF5 users to recognize exposure surfaces, because many privacy failures arise during legitimate sharing and reuse rather than from malicious access.

- Privacy guidance for HDF5 should be aligned with open-science goals: the objective is not to make data opaque by default, but to make exposure intentional, reviewable, and proportionate to the release context.

4 HDF5 Extensions

4.1 Overview

Supply chain-related issues are discussed in more detail in [Section 6](#).

4.2 Virtual Object Layer (VOL) Connectors

4.3 Filters Plugins

4.4 Virtual File Drivers (VFD)

4.5 Audit Findings and Mitigation Priorities

5 HDF5 Wrappers and Language Bindings

5.1 Overview

Complete:

- C#
- Java
- Python
- JavaScript
- Rust

Partials...

5.2 Python

- CVE-2025-995 [[19](#)]
- CVE-2026-0897 [[20](#)]
- CVE-2026-1669 [[21](#)]
- CVE-2026-2492 [[15](#)]

5.3 Java

5.4 Audit Findings and Mitigation Priorities

6 HDF5 in the Software Supply Chain

- HDF5 2.0.0 source-artifact integrity check [28]
- SBOM

6.1 Upstream Dependencies

This subsection summarizes external software that can be required by an HDF5 build. It is organized by build option, because the default serial C library has a much smaller dependency set than feature-complete, testing, packaging, or developer builds.

6.1.1 Dependency Source Modes

HDF5’s CMake build can either locate dependencies that are already installed or, for selected filter-related projects, add external source trees to the build with `FetchContent`.

Mode	Relevant options	Meaning
System packages	HDF5_ALLOW_EXTERNAL_SUPPORT=NO	CMake uses <code>find_package()</code> , <code>find_library()</code> , or <code>find_program()</code> to locate installed dependencies. This is the default.
Git fetch	HDF5_ALLOW_EXTERNAL_SUPPORT=GIT plus a dependency-specific *_USE_EXTERNAL=ON option	CMake downloads source from the configured <code>*_GIT_URL</code> and <code>*_GIT_TAG</code> values and builds it with HDF5.

Tarball fetch	HDF5_ALLOW_EXTERNAL_SUPPORT=	CMake downloads from
	TGZ plus a dependency-specific	*_TGZ_ORIGPATH
	*_USE_ uses	*_TGZ_NAME, or
	EXTERNAL=ON option	TGZPATH when
		the dependency-specific
		*_USE_LOCALCONTENT=ON
		option is set.

The HDF5_ALLOW_EXTERNAL_SUPPORT modes apply to filter-related dependencies. Some test and example configurations have their own source-fetch paths, such as KWSys for the API test driver and h5py for Python examples. MPI, HDF5, ROS3, OpenSSL, Java, documentation, packaging, and developer tools must be provided by the build environment.

6.1.2 Baseline Build

Dependency	Trigger	Notes
CMake 3.26 or newer	Always	Required by the top-level cmake_minimum_required().
C compiler and native build tool	Always	The top-level project is a C project.
Platform C library, headers, and system calls	Always	CMake probes standard headers, types, functions, byte order, large-file support, file locking, pread/pwrite, and related platform features.
Platform link libraries	Platform-dependent	CMake adds libraries only when probes show they are needed: m, dl, rt, posix4, ucb, ws2_32, wsock32, and shlwapi.
PkgConfig	Optional probe	Used only when available. It can help some find modules, such as zlib-ng and libaec, but it is not a hard requirement for the default build.

Threads	Found in every configuration; required by thread features	CMake searches for C11 threads, Win32 threads, or pthreads. When found, HDF5 records thread support and links <code>Threads::Threads</code> ; <code>HDF5_ENABLE_THREADSAFE</code> , <code>HDF5_ENABLE_CONCURRENCY</code> , and <code>HDF5_ENABLE_SUBFILING_VFD</code> require it.
---------	---	---

6.1.3 Feature Dependencies

Feature or component	Build options	Dependency	Notes
Parallel HDF5	<code>HDF5_ENABLE_PARALLEL=ON</code>	MPI C with MPI-IO, MPI standard 3.0 or newer	Adds <code>MPI::MPI_C</code> to public link libraries. Parallel filtered writes and large parallel I/O are enabled only when the required MPI symbols are present. Parallel tests also need an MPI launcher such as <code>mpiexec</code> .
Parallel Fortran	<code>HDF5_ENABLE_PARALLEL=ON</code> , <code>HDF5_BUILD_FORTRAN=ON</code>	MPI Fortran	The Fortran build adds MPI Fortran libraries. The <code>mpi_f08</code> module is used when available.

Thread-safe library	HDF5_ENABLE_THREADSAFE=ON	Threads	Unsupported with parallel, Fortran, C++, or high-level library unless HDF5_ALLOW_UNSUPPORTED=ON. On Windows, static libraries are rejected.
Multi-threaded concurrency	HDF5_ENABLE_CONCURRENCY=ON	Threads	Experimental. Mutually exclusive with HDF5_ENABLE_THREADSAFE; has the same unsupported combinations as thread-safety unless overridden with HDF5_ALLOW_UNSUPPORTED=ON.
C++ wrappers	HDF5_BUILD_CPP_LIB=ON	C++ compiler	Unsupported with parallel unless HDF5_ALLOW_UNSUPPORTED=ON. High-level C++ wrappers also require HDF5_BUILD_HL_LIB=ON.
Fortran wrappers	HDF5_BUILD_FORTRAN=ON	Fortran compiler with Fortran 2003 ISO_C_BINDING support	Unsupported with thread-safety or concurrency unless HDF5_ALLOW_UNSUPPORTED=ON. High-level Fortran wrappers also require HDF5_BUILD_HL_LIB=ON.

Java wrappers	HDF5_BUILD_ JAVA=ON	Java/JDK, shared HDF5 libraries, Java dependency JARs	Java requires shared HDF5 libraries. Java 11 or newer is re- quired for the JNI im- plementation. Java tests additionally re- quire the JUnit and Hamcrest JARs.
Java JNI imple- mentation	HDF5_BUILD_ JAVA=ON, HDF5_ ENABLE_JNI=ON, or any Java version earlier than 25	JNI headers and libraries	JNI is the default Java implementation. HDFS also requires JNI.
Java FFM imple- mentation	HDF5_BUILD_ JAVA=ON, Java 25 or newer, HDF5_ ENABLE_JNI=OFF	jextract from JEXTRACT_ HOME/bin or JAVA_HOME/ bin	Generates FFM bindings at configure time.
Java logging/test JARs	HDF5_BUILD_ JAVA=ON	slf4j-api, slf4j-nop, slf4j-simple; plus JUnit 4 and Hamcrest for tests	The default paths point to bun- dled JARs under java/lib. They can be overrid- den with HDF5_ JAVA_*_JAR cache variables. HDF5_ ENABLE_MAVEN_ DEPLOY=ON gen- erates Maven-style artifacts and POM files; the CMake build does not locate or run mvn.

Documentation	HDF5_BUILD_DOC=ON	Doxygen	HDF5_ENABLE_DOXY_WARNINGS=ON makes Doxygen warnings fail the build.
Header regeneration	HDF5_GENERATE_HEADERS=ON	Perl	Perl is optional otherwise. If it is missing, generated headers are not regenerated.
Python examples	H5EXAMPLE_BUILD_PYTHON=ON	Python3 interpreter, Python development files, NumPy, h5py source, pip	The examples CMake project fetches h5py from Git and installs it with python3 -m pip.

6.1.4 Filter Dependencies

Feature	Build options	Dependency	Source choices
DEFLATE filter	HDF5_ENABLE_ZLIB_SUPPORT=ON, HDF5_USE_ZLIB_NG=OFF	zlib	System package by default; ZLIB_USE_EXTERNAL=ON can build zlib from Git or TGZ. HDF5_USE_ZLIB_STATIC=ON prefers a static library.

DEFLATE filter through zlib-ng	HDF5_ENABLE_ZLIB_SUPPORT=ON, HDF5_USE_ZLIB_NG=ON	zlib-ng	System package by default; ZLIB_USE_EXTERNAL=ON can build zlib-ng from Git or TGZ. zlib compatibility mode is detected.
SZIP filter	HDF5_ENABLE_SZIP_SUPPORT=ON	libaec with libsz compatibility, or another package selected by LIBAEC_PACKAGE_NAME	System package by default; SZIP_USE_EXTERNAL=ON can build libaec from Git or TGZ. HDF5_USE_LIBAEC_STATIC=ON prefers static libraries.
HDF5 filter plugins project	HDF5_ENABLE_PLUGIN_SUPPORT=ON	Installed h5pl/HDF5 filter plugins package, or the hdf5_plugins project	PLUGIN_USE_EXTERNAL=ON can build the plugin project from Git or TGZ. Plugin support is disabled for the 1.6 API.

The plugin project can introduce additional filter-library dependencies. The HDF5 cache defaults include entries for bitroom, bitround, bitshuffle, Blosc, Blosc2, bzip2, fpzip, JPEG, LZ4, LZF, SZ, ZFP, and Zstandard. Those libraries are controlled by the plugin project’s own options when hdf5_plugins is built or found.

6.1.5 Virtual File Drivers and External Connectors

Feature	Build options	Dependency	Notes
Direct VFD	HDF5_ENABLE_ DIRECT_VFD=ON	POSIX O_DIRECT and posix_ memalign() support	No third-party library. Configuration fails if the platform support is missing.
Mirror VFD	HDF5_ENABLE_ MIRROR_VFD=ON	POSIX socket/fork headers and func- tions	No third-party library. Requires netinet/in.h, netdb.h, arpa/inet.h, sys/socket.h, and fork().
ROS3 VFD	HDF5_ENABLE_ ROS3_VFD=ON	aws-c-s3 CMake package	HDF5 does not fetch this library. Building aws-c-s3 from source can also require AWS C libraries such as aws-c-common, aws-checksums, aws-c-cal, aws-c-io, aws-c-compression, aws-c-http, aws-c-sdkutils, aws-c-auth, and, on Linux, aws-lc and s2n-tls. ROS3 docker-proxy tests additionally require docker and the AWS CLI.

HDFS VFD	HDF5_ENABLE_ HDFS=ON	JNI/JVM and Hadoop libhdfs	is used to locate hdfs.h, libhdfs, and the Hadoop version command. Non-MSVC builds also add -pthread.
Subfiling VFD and IOC VFD	HDF5_ENABLE_ PARALLEL=ON, HDF5_ENABLE_ SUBFILING_VFD=ON	MPI C with MPI_Comm_split_type, plus Threads	Not available on Windows. IOC VFD is built when subfiling is enabled.
External VOL connectors	HDF5_VOL_ALLOW_ EXTERNAL=GIT or LOCAL_DIR with HDF5_VOL_URLnn or HDF5_VOL_PATHnn	Connector-defined	Up to ten CMake-based VOL connectors can be added. HDF5 patches connector CMake code to use the in-tree HDF5 build, but each connector may bring its own dependencies.

6.1.6 Tools, Tests, and Security Features

Feature	Build options	Dependency	Notes
Signed plugin verification	HDF5_REQUIRE_ SIGNED_PLUGINS= ON	OpenSSL libcrypto 1.1.0 or newer	Adds OpenSSL::Crypto to HDF5. HDF5_LOCK_PLUGIN_KEYSTORE=ON also requires HDF5_PLUGIN_KEYSTORE_DIR.

h5sign tool	HDF5_REQUIRE_ SIGNED_PLUGINS= ON, HDF5_BUILD_ TOOLS=ON	OpenSSL libcrypto	Used to sign HDF5 plugins.
Signed-plugin tests	HDF5_REQUIRE_ SIGNED_PLUGINS= ON, BUILD_TESTING= ON	openssl exe- cutable	Tests generate RSA keys with the OpenSSL command-line tool.
Parallel tools utili- ties	HDF5_ENABLE_ PARALLEL=ON, HDF5_ BUILD_PARALLEL_ TOOLS=ON	MFU, CIRCLE, CMake DTCMP	looks for headers and libraries, using MFU_ROOT as a hint.
API test driver	BUILD_TESTING=ON, HDF5_TEST_API_ ENABLE_DRIVER=ON	KWSys	The test driver fetches KWSys from a tar- ball or from TGZPATH when KWSYS_USE_ LOCALCONTENT= ON.
Shell-script tests	BUILD_TESTING=ON	PowerShell or Bash	CMake uses pwsh/powershell when available; otherwise Unix builds look for bash.
ROS3 VFD tests	HDF5_ENABLE_ROS3_ VFD=ON, HDF5_ ENABLE_ROS3_VFD_ DOCKER_PROXY=ON	Docker daemon and AWS CLI	Used to run S3 proxy tests.

6.1.7 Developer and Packaging Dependencies

Configuration	Build options	Dependency
Analyzer tooling	HDF5_ENABLE_ ANALYZER_TOOLS=ON	clang-tidy, include-what-you-use, and cppcheck are probed and used by the analyzer macros.
Source formatting tar- gets	HDF5_ENABLE_ FORMATTERS=ON	clang-format and cmake-format.
Code coverage with Clang/IntelLLVM	HDF5_ENABLE_ COVERAGE=ON, CODE_COVERAGE=ON	llvm-cov and llvm-profdata; Ap- pleClang uses xcrun llvm-cov and xcrun llvm-profdata.
Code coverage with GCC	HDF5_ENABLE_ COVERAGE=ON, CODE_COVERAGE=ON	lcov and genhtml; fall- back instrumentation can also link gcov.
Sanitizers	HDF5_ENABLE_ SANITIZERS=ON, HDF5_USE_SANITIZER= <kind>	Compiler sanitizer runtime support. Supported values depend on compiler and plat- form.
AFL support module	Explicit inclusion of config/sanitizer/ afl-fuzzing.cmake	afl-cc and afl-c++. This module is available but is not included by the top- level HDF5 build by default.
Dependency graph support module	Explicit inclusion of config/sanitizer/ dependency-graph. cmake	Graphviz dot. This mod- ule is available but is not included by the top-level HDF5 build by default.
Windows installers	CPack on Windows	NSIS and WiX are probed when available.
Linux packages	CPack on Unix-like systems	dpkg-shlibdeps en- ables DEB generation; rpmbuild enables RPM generation for non-parallel builds.

6.1.8 External Source Defaults

The following default upstream source locations are encoded in `config/CacheURLs`. `cmake` for dependencies that HDF5 can fetch or whose values are passed to external filter-plugin builds.

Dependency	Default sion/tag	ver- Default source
zlib	1.3.2 / v1.3.2	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.2/zlib-1.3.2.tar.gz
zlib-ng	2.3.3	https://github.com/zlib-ng/zlib-ng.git or https://github.com/zlib-ng/zlib-ng/archive/refs/tags/2.3.3.tar.gz
libaec	1.1.6 / v1.1.6	https://github.com/MathisRosenhauer/libaec.git or https://github.com/MathisRosenhauer/libaec/releases/download/v1.1.6/libaec-1.1.6.tar.gz
HDF5 filter plugins	master	https://github.com/HDFGroup/hdf5_plugins.git or https://github.com/HDFGroup/hdf5_plugins/releases/download/snapshot/hdf5_plugins-master.tar.gz
KWSys API-test helper	master tarball	https://gitlab.kitware.com/utils/kwsys/-/archive/master/kwsys-master.tar.gz
h5py Python examples	ex-master	https://github.com/h5py/h5py.git

bitshuffle	master or 0.5.2 tarball	https://github.com/kiyo-masui/bitshuffle.git or https://github.com/kiyo-masui/bitshuffle/archive/refs/tags/bitshuffle-0.5.2.tar.gz
Blosc	1.21.6	https://github.com/Blosc/c-blosc.git or https://github.com/Blosc/c-blosc/archive/refs/tags/c-blosc-1.21.6.tar.gz
Blosc zlib helper	develop or 1.3.1 tarball	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.1/zlib-1.3.1.tar.gz
Blosc2	2.17.1	https://github.com/Blosc/c-blosc2.git or https://github.com/Blosc/c-blosc2/archive/refs/tags/c-blosc2-2.17.1.tar.gz
Blosc2 zlib helper	develop or 1.3.1 tarball	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.1/zlib-1.3.1.tar.gz
bzip2	bzip2-1.0.8	https://github.com/libarchive/bzip2.git or https://github.com/libarchive/bzip2/archive/refs/tags/bzip2-bzip2-1.0.8.tar.gz
fpzip	develop or 1.3.0 tarball	https://github.com/LLNL/fpzip.git or https://github.com/LLNL/fpzip/releases/download/1.3.0/fpzip-1.3.0.tar.gz
JPEG	jpeg-9e	https://github.com/libjpeg-turbo/libjpeg-turbo.git or https://www.ijg.org/files/jpegsrc.v9e.tar.gz

LZ4	dev or 1.10.0 tarball	https://github.com/lz4/lz4 . git or https://github.com/lz4/ lz4/releases/download/v1.10.0/ lz4-1.10.0.tar.gz
LZF	3.6 tarball	http://dist.schmorp.de/liblzf/ liblzf-3.6.tar.gz
SZ	master or 2. 1.12.5 tarball	https://github.com/ szcompressor/SZ .git or https://github.com/szcompressor/SZ/ releases/download/v2.1.12.5/ SZ-2.1.12.5.tar.gz
ZFP	develop or 1. 0.1 tarball	https://github.com/LLNL/zfp . git or https://github.com/LLNL/ zfp/releases/download/1.0.0/ zfp-1.0.1.tar.gz
Zstandard	1.5.7	https://github.com/facebook/ zstd .git or https://github. com/facebook/zstd/releases/ download/v1.5.7/zstd-1.5.7. tar.gz

6.2 Downstream Dependencies

For this audit, downstream dependencies are projects that compile against, dynamically load, package, or define data conventions on top of HDF5. The HDF5 ecosystem is much larger than any static list, so the most important downstreams are best identified by blast radius: broad installation footprint, long-lived archival data, domain-standard file conventions, or use as an intermediate layer for other tools.

6.2.1 High-Impact Downstream Families

Scientific data models and exchange formats.

The highest-impact downstream is netCDF-4, which uses HDF5 as the storage

layer for the enhanced netCDF data model and is central to climate, weather, ocean, atmospheric, and other Earth-system data workflows [29]. NASA HDF-EOS5, NeXus, and CGNS similarly define domain-specific conventions over HDF5 for Earth observation, neutron/x-ray/muon facilities, and CFD or aerospace simulation data [14, 23, 7]. For these formats, HDF5 compatibility is not just a library question: it affects the readability of long-lived archives and the ability of independent tools to interpret a shared schema.

Language bindings and analysis libraries.

h5py, PyTables, pandas HDFStore, Bioconductor's rhdf5, HDF5.jl, and MATLAB's HDF5 interfaces make HDF5 a routine dependency for Python, R, Julia, and MATLAB users [13, 26, 25, 6, 17, 22]. These packages are important because they often sit at the boundary between upstream HDF5 builds and end-user scripts. Their packaging choices determine whether users load a bundled HDF5, a system HDF5, or an environment-provided HDF5, and therefore whether ABI, plugin-path, thread-safety, MPI, and filter support match the files being opened.

Geospatial, visualization, and inspection tools.

GDAL's HDF5 driver, ParaView/VTK-family builds, ITK's HDF5 module, HDFView, and similar readers expose HDF5 files to interactive and automated analysis workflows [24, 18, 16]. This is an important security boundary: these tools are commonly used to inspect third-party data files, so robustness against malformed HDF5 objects, unusual filters, and unexpected VFD/plugin configuration has direct operational value.

HPC and parallel-I/O frameworks.

ADIOS2 includes an HDF5 engine, while netCDF-4, CGNS, h5py, and many simulation applications can be built against serial or parallel HDF5 variants [1]. These downstreams amplify configuration risk: MPI ABI compatibility, collective I/O semantics, file locking behavior, subfiling support, and optional VFDs must line up across the application, HDF5, MPI, and filesystem stack.

Machine-learning and model-artifact workflows.

TensorFlow/Keras still supports older HDF5 model and weight files even as newer native formats are preferred [27]. This keeps HDF5 in the model-

exchange and training-checkpoint ecosystem, usually through h5py, and makes HDF5 parser behavior relevant for model files obtained from external sources.

6.2.2 Representative Downstream Projects

Downstream	Role	Supply-chain relevance
netCDF-4	Common scientific data model and format layered on HDF5.	Large transitive footprint through climate, weather, ocean, remote-sensing, and HPC data stacks. Uses a constrained HDF5 subset, so compatibility depends on both HDF5 behavior and netCDF conventions.
HDF-EOS5	NASA Earth-observation format built on HDF5 concepts and libraries.	Long-lived public archives and many third-party readers make backward compatibility and robust metadata handling high-value.
NeXus	Facility data convention for neutron, x-ray, and muon science.	Uses HDF5 groups, datasets, and attributes as a standardized experimental-data container, so HDF5 schema interpretation affects reproducibility across facilities.
CGNS	CFD and aerospace data standard with an HDF5 mapping.	Important for mesh and simulation-result exchange; parallel-HDF5 behavior and large-file support affect production simulation workflows.

h5py, PyTables, and pandas HDFStore	Python interfaces and tabular/array storage layers.	Common bridge between binary HDF5 files and Python analysis. Wheel and conda packaging can hide which HDF5 build is actually loaded.
rhdf5, HDF5.jl, MATLAB HDF5 APIs	R, Julia, and MATLAB access layers.	Expose HDF5 to laboratory and scientific-computing workflows where files are often exchanged outside controlled build environments.
GDAL and geospatial readers	Raster and subdataset access for HDF5-based products.	Converts HDF5 issues into geospatial ingestion issues, especially for remote-sensing products with complex metadata and nested subdatasets.
ParaView/VTK, ITK, HDFView, and viewers	Interactive inspection, visualization, and conversion.	Frequently open untrusted or externally generated files; defensive parsing, filter availability, and plugin search paths matter.
ADIOS2 and simulation I/O layers	Alternative I/O stacks with HDF5 read/write engines or compatibility paths.	Parallel and high-throughput workflows depend on matching MPI, filesystem, and HDF5 feature assumptions across the whole stack.
TensorFlow/Keras HDF5 artifacts	Legacy model and weight serialization.	Keeps HDF5 in ML artifact exchange; malformed or incompatible model files surface through h5py and HDF5 library behavior.

6.2.3 Audit Implications

- HDF5 changes have a large transitive blast radius because many users reach HDF5 through netCDF, GDAL, h5py, MATLAB, or domain formats rather than through the HDF5 C API directly.

- ABI and packaging mismatches are a recurring downstream risk. Python, R, Julia, MATLAB, MPI, and system-package environments may all load different HDF5 builds in the same organization.
- File parsing is a practical security boundary. Viewers, converters, notebooks, and data-ingestion jobs often process files received from outside the organization that built the HDF5 library.
- Optional features can become hidden interoperability requirements: filter plugins, SZIP/zlib variants, ROS3/HDFS/VFD support, parallel HDF5, and file-locking behavior may be assumed by downstream data even when they are not present in a default HDF5 build.
- Long-lived scientific archives make deprecation slower than ordinary application software. Even when a downstream project prefers a newer format, historic HDF5 files remain operationally relevant.

6.3 Audit Findings and Mitigation Priorities

7 Independent HDF5 Implementations

7.1 Overview

- HDF5 library playing footsie with the file format spec: GitHub issue 6409 [2]

PureHDF Managed .NET read/write without native binaries

jHDF JVM HDF5 reading

netCDF-Java JVM netCDF/HDF-EOS/HDF5 scientific-data reading

pyfive Pure Python read-only inspection

jsfive Browser-side direct HDF5 read

scigolib/hdf5 Pure Go read/write

hdf5-pure and rustyhdf5-format/rustyhdf5 Pure Rust read/write

async-hdf5 or h5coro Cloud object-store chunk mapping

JLD2.jl Julia-native persistence in HDF5-compatible files

7.2 Audit Findings and Mitigation Priorities

The main risk across all direct-format implementations is not opening simple files; it is coverage of the long tail: dense groups, fractal heaps, shared object-header messages, all chunk-index variants, references, VLEN, compound/nested datatypes, virtual datasets, external storage, SWMR flags, user blocks, non-core filters, unusual libver bounds, and files produced by different HDF5 library releases. For any production use, build a compatibility corpus and round-trip the generated files using `h5dump`, `h5debug`, `h5py/libhdf5`, and the target-independent implementation.

Machine-readable specification: `hdf5-pickles`

8 Long-term Data Accessibility and Interpretability

Long term data accessibility and interpretability [4]

8.1 Audit Findings and Mitigation Priorities

9 Resources

- HDF SSP SIG [[12](#)]
- Tools
 - HDF5 CVE test suite [[9](#)]
 - A machine-readable specification of the HDF5 file format. [[8](#)]
 - HDF5 extension templates [[10](#)]
 - HDF5 plugins [[11](#)]
- S2D2

References

- [1] ADIOS2 Developers. *ADIOS2 Supported Engines: HDF5*. 2026. URL: <https://adios2.readthedocs.io/en/latest/engines/engines.html#hdf5>. (accessed: 29.05.2026).
- [2] Apollo3zehn. *HDF5 lib is more restrictive than file format spec*. 2026. URL: <https://github.com/HDFGroup/hdf5/issues/6409>. (accessed: 21.05.2026).
- [3] Multiple authors. *179 results for 'file corruption'*. 2007-2026. URL: <https://forum.hdfgroup.org/search?q=file%20corruption>. (accessed: 21.05.2026).
- [4] Multiple authors. *223 results for 'plugin'*. 2007-2026. URL: <https://forum.hdfgroup.org/search?q=plugin>. (accessed: 21.05.2026).
- [5] Andreas Beckmann. *How to properly close HDF files when disk is full?* 2023. URL: <https://forum.hdfgroup.org/t/how-to-properly-close-hdf-files-when-disk-is-full/11714>. (accessed: 21.05.2026).
- [6] Bioconductor Project. *rhdf5*. 2026. URL: <https://bioconductor.org/packages/rhdf5>. (accessed: 29.05.2026).
- [7] CGNS Steering Committee. *CGNS/HDF5 – An HPC implementation*. 2026. URL: <https://cgns.org/standard/hdf5.html>. (accessed: 29.05.2026).
- [8] The HDF Group. *A machine-readable specification of the HDF5 file format*. 2026. URL: <https://github.com/HDFGroup/hdf5-pickles>. (accessed: 21.05.2026).
- [9] The HDF Group. *HDF5 CVE Test Suite*. 2026. URL: https://github.com/HDFGroup/cve_hdf5. (accessed: 21.05.2026).
- [10] The HDF Group. *HDF5 extension skeletons, literally*. 2026. URL: <https://github.com/HDFGroup/hdf5-boneyard>. (accessed: 21.05.2026).
- [11] The HDF Group. *HDF5 Plugins*. 2026. URL: https://github.com/HDFGroup/hdf5_plugins. (accessed: 21.05.2026).

- [12] The HDF Group. *HDF5 Safety, Security & Privacy (SSP) Special Interest Group's governance, processes, and artifacts*. 2026. URL: <https://github.com/HDFGroup/hdf5-ssp-sig>. (accessed: 21.05.2026).
- [13] h5py Developers. *HDF5 for Python*. 2026. URL: <https://docs.h5py.org/>. (accessed: 29.05.2026).
- [14] HDF-EOS Tools and Information Center. *HDF-EOS Libraries*. 2026. URL: <https://hdfeos.org/software/library.php>. (accessed: 29.05.2026).
- [15] Zero Day Initiative. *CVE-2026-2492*. 2026. URL: <https://www.cve.org/CVERecord?id=CVE-2026-2492>. (accessed: 21.05.2026).
- [16] Insight Software Consortium. *ITK Module: ITKHDF5*. 2026. URL: https://docs.itk.org/projects/doxygen/en/stable/group__ITKHDF5.html. (accessed: 29.05.2026).
- [17] JuliaIO. *HDF5.jl*. 2026. URL: <https://juliaio.github.io/HDF5.jl/dev/>. (accessed: 29.05.2026).
- [18] Kitware, Inc. *Building ParaView*. 2026. URL: https://www.paraview.org/paraview-docs/latest/cxx/md__builds_gitlab-kitware-sciviz-ci_Documentation_dev_build.html. (accessed: 29.05.2026).
- [19] Google LLC. *CVE-2025-9905*. 2025. URL: <https://www.cve.org/CVERecord?id=CVE-2025-99057>. (accessed: 21.05.2026).
- [20] Google LLC. *CVE-2026-0897*. 2026. URL: <https://www.cve.org/CVERecord?id=CVE-2026-0897>. (accessed: 21.05.2026).
- [21] Google LLC. *CVE-2026-1669*. 2026. URL: <https://www.cve.org/CVERecord?id=CVE-2026-1669>. (accessed: 21.05.2026).
- [22] MathWorks. *HDF5 Files*. 2026. URL: <https://www.mathworks.com/help/matlab/hdf5-files.html>. (accessed: 29.05.2026).
- [23] NeXus International Advisory Committee. *NeXus Data Format Manual*. 2026. URL: <https://manual.nexusformat.org/>. (accessed: 29.05.2026).

- [24] Open Source Geospatial Foundation. *HDF5 – Hierarchical Data Format Release 5 (HDF5)*. 2026. URL: <https://gdal.org/en/stable/drivers/raster/hdf5.html>. (accessed: 29.05.2026).
- [25] pandas Developers. *IO tools: HDF5 / PyTables*. 2026. URL: https://pandas.pydata.org/docs/dev/user_guide/io.html#hdf5-pytables. (accessed: 29.05.2026).
- [26] PyTables Developers. *PyTables Documentation*. 2026. URL: <https://www.pytables.org/>. (accessed: 29.05.2026).
- [27] TensorFlow Authors. *Save, serialize, and export models*. 2026. URL: https://www.tensorflow.org/guide/keras/save_and_serialize. (accessed: 29.05.2026).
- [28] The HDF Group. *HDF5 2.0.0 source-artifact integrity check*. URL: <https://github.com/HDFGroup/hdf5-ssp-sig/tree/main/audit/proofs/hdf5-core/2.0.0/psirt/release-integrity/PRISUPP-2204>. (accessed: 03.05.2026).
- [29] Unidata Program Center. *NetCDF-4 File Format*. 2026. URL: https://docs.unidata.ucar.edu/nug/2.0-draft/nc4_file_format.html. (accessed: 29.05.2026).